FEAT-ZD-48600 · BE-04 Identify the components/layers to be changed · Components, layers, sources or pipelines to change

Backend agent · item task.components · source hobasa-backend-agent/outputs/FEAT-ZD-48600/task.components.md · approved by automated checks and independent judge

task.components: FEAT-ZD-48600

Components and layers the Backend lane must create or update for FEAT-ZD-48600 (ZD-48600, "Show which overdue invoices belong to vendors that usually pay late"), kept to the minimum lines of code. This is BE-04 Identify the components/layers to be changed (attempt 1). No gate: automated checks, the independent judge and sampled human evaluation.

Result: one repository (ap_aging), two source files changed and one new test file. No new module, layer, service, transport, storage or dependency. Everything else in ap_aging stays as it is.

Field	Value	Source
Trace ID	FEAT-ZD-48600	record:brief
Step	BE-04 Identify the components/layers to be changed (attempt 1)	record:brief
Owner / gate	Developer / automated checks, independent judge, sampled human evaluation	record:brief
Inputs	feat.tasks (b0fd4d2), feat.stories (0590ea4), feat.prd (908b58d), feat.trd (3637e51), feat.design (722a98f), task.repo_access (c7b4be8)	record:brief
Background read	task.db_changes (BE-03): no database change is required	journey:task.db_changes
Code read	`src/ap_aging/__init__.py`, `src/ap_aging/aging.py`, `tests/__init__.py`, `tests/test_aging.py`	tool:repo
Backend tasks in scope	TSK-05 (STORY-05, `overdue_with_risk`) and TSK-06 (STORY-06, `overdue_rows_to_json`); TSK-06 depends on TSK-05	feat.tasks (section 1)
Repository and branch	ap_aging, branch `feature/story` at `5d2cbb9`, write access confirmed	task.repo_access (section 1)

1. What the existing code does today

Element	What it is	What this feature does with it	Source
`aging.py` imports	`from datetime import date`, `from decimal import Decimal`, `from typing import Dict, Iterable`	Reuse all three; add `List` to the `typing` import and add `import json`	tool:repo; feat.design (B3.1)
`BUCKETS`	Tuple of the five bucket labels	Unchanged; not used by the new code	tool:repo; feat.tasks (TSK-05 "Must not change")
`days_past_due(due_date, as_of)`	`max((as_of - due_date).days, 0)`: calendar days after the due date, 0 when not yet due	**Reused** for `days` in the overdue list; no new day arithmetic	tool:repo; feat.trd (IF-5.2); record:memory (D-2026-031)

Element	What it is	What this feature does with it	Source
`bucket_for(days)`	Returns `"1-30"` only when `days < 30`, so `bucket_for(30)` is `"31-60"`	Unchanged and **never called** by the new code (DD-4)	tool:repo; feat.trd (F-10, IF-5.3); feat.design (DD-4)
`aging_buckets(invoices, as_of)`	Skips `inv.get("paid")` truthy; converts `Decimal(inv["amount"])`	Unchanged. The new code copies its paid test and its `Decimal(...)` conversion	tool:repo; feat.trd (IF-5.1, IF-5.5)
`__init__.py`	`from .aging import aging_buckets; __version__ = "1.4.0"; __all__ = ["aging_buckets"]`	Import and export the two new functions; one version step	tool:repo; feat.design (B3.1)
`tests/test_aging.py`	`AS_OF = date(2026, 10, 25)`; tests `days_past_due`, `bucket_for` edges (0, 31, 90, 91) and `aging_buckets`	Unchanged. It does not pin `bucket_for(30)`, so the regression pin goes into the new test file	tool:repo; feat.tasks (TSK-05 "Tests to add")
Layers present	One library module (`aging.py`) behind a package API (`__init__.py`). No service, transport or database layer	No layer is added: CT-1 is a string returned by a function (DD-3) and no database change is needed	tool:repo; feat.design (A3 "Transport for CT-1", DD-3); journey:task.db_changes

2. Components to create or update

#	Layer	File	Change	Kind	Task / story	Source
BC-1	Domain logic	`src/ap_aging/aging.py`	Add `import json`; add `List` to the existing `typing` import	Update (imports)	TSK-05, TSK-06	feat.design (B3.1); feat.tasks (TSK-05, TSK-06 "Files in budget")
BC-2	Domain logic	`src/ap_aging/aging.py`	Add module constants `OVERDUE_AFTER_DAYS = 30` and `TWO_PLACES = Decimal("0.01")`	Create	TSK-05	feat.design (B3.2, DD-4, DD-10); feat.tasks (TSK-05)
BC-3	Domain logic	`src/ap_aging/aging.py`	Add `overdue_with_risk(invoices: Iterable[Dict], as_of: date, risk_by_vendor: Dict[str, str]) -> List[Dict]` (HC-6)	Create (new function)	TSK-05 / STORY-05	feat.design (HC-6, B3.2); feat.trd (C-6, IF-5)
BC-4	Domain logic (serialisation)	`src/ap_aging/aging.py`	Add `overdue_rows_to_json(rows: Iterable[Dict]) -> str` (HC-7, contract CT-1)	Create (new function)	TSK-06 / STORY-06	feat.design (HC-7, CT-1, B3.2); feat.trd (C-7, IF-6)
BC-5	Package API	`src/ap_aging/__init__.py`	Import `overdue_with_risk` and `overdue_rows_to_json` from `.aging`; add both to `__all__`	Update	TSK-05, TSK-06	feat.design (B3.1); feat.trd (F-11)

#	Layer	File	Change	Kind	Task / story	Source
BC-6	Package version	`src/ap_aging/__init__.py`	One minor version step from `1.4.0`, owned by TSK-05; the lane confirms the next number	Update	TSK-05	feat.tasks (section 2 "Version step"); feat.design (DD-11)
BC-7	Unit tests	New `tests/test_overdue.py`	Tests for BC-3 and BC-4 plus the regression pin on the existing functions (section 4)	Create	TSK-05, TSK-06	feat.tasks (section 2 "Test files"; TSK-05, TSK-06 "Tests to add")

Placement follows the approved design: both functions go into `aging.py`, next to `days_past_due`, which they reuse (feat.trd T-A4, C-6; feat.design DD-2).

3. Minimum-lines approach for each component

The aim is the smallest change that meets every acceptance criterion. The choices below are agent judgement within the approved design; none changes a contract.

#	Minimum-lines choice	Why it is enough	Source
ML-1	Reuse `days_past_due` instead of computing days again	It already counts calendar days and returns 0 when not yet due, which is what IF-5.2 and AC-05.4 need	tool:repo; feat.trd (IF-5.2); feat.stories (AC-05.1, AC-05.4)
ML-2	Copy `aging_buckets`'s paid test, `inv.get("paid")`, as the first check	Same rule as today (D-2026-050), and the paid skip happens before `vendor` is read	tool:repo; feat.trd (IF-5.1); record:memory (D-2026-050)
ML-3	One module constant compared with > (`days > OVERDUE_AFTER_DAYS`); no call to `bucket_for`	Excludes exactly 30 days (AC-05.2) without touching `bucket_for`	feat.design (DD-4); feat.trd (F-10, IF-5.3); record:memory (L-007)
ML-4	Build the row inline as a five-key dict; `risk_by_vendor.get(inv["vendor"], "unknown")`	Gives the exact keys (AC-05.5) and `'unknown'` (AC-05.7) with no helper; `inv["vendor"]` with no default raises `KeyError` (AC-05.10)	feat.trd (IF-5.4, IF-5.7); feat.design (B3.2 step 3); feat.stories (AC-05.5, AC-05.7, AC-05.10)
ML-5	`Decimal(inv["amount"]).quantize(TWO_PLACES)` with the default context	Keeps `Decimal`, never `float` (AC-05.8); same conversion as `aging_buckets`	feat.trd (IF-5.5); feat.design (DD-10); record:memory (D-2026-044, L-011)
ML-6	One in-place sort of the new list: `rows.sort(key=lambda row: (-row["days"], row["id"]))`	Days descending, then `id` ascending as text (AC-05.6); the input is not changed	feat.trd (IF-5.6); feat.design (B3.2 step 4, NF-10)
ML-7	`overdue_rows_to_json` is a single `json.dumps` over a list comprehension, sharing `TWO_PLACES`	CT-1 key order comes from dict insertion order; `amount` becomes `str(Decimal(r["amount"]).quantize(TWO_PLACES))`; `days` becomes `int(r["days"])`; `[]` for no rows; no `float`	feat.design (B3.2); feat.trd (IF-6); feat.tasks (TSK-06 "Implementation notes")
ML-8	No private helper, no new module, no class	Both functions are short enough to read inline; a new module would add import and export plumbing for no gain	feat.design (DD-2); feat.tasks (section 2 size budget S and M)

#	Minimum-lines choice	Why it is enough	Source
ML-9	Leave docstrings of the existing module and package unchanged; give each new function the docstring the design gives it	Keeps the diff to new code only, and documents each new function as designed	feat.design (B3.2 docstrings)

Sketch of the additions to `aging.py`. The signatures, constants and docstrings are the design's; the loop body is assembled here from the design's numbered algorithm (steps 1 to 4). The coding step writes the final code.

```
import json
from typing import Dict, Iterable, List   # List added

OVERDUE_AFTER_DAYS = 30
TWO_PLACES = Decimal("0.01")

def overdue_with_risk(invoices: Iterable[Dict], as_of: date, risk_by_vendor: Dict[str, str]) -> List[Dict]:
    """Unpaid invoices more than 30 calendar days past due as of as_of, as rows
    {'id', 'vendor', 'days', 'amount' (Decimal, 2 dp), 'risk'}, sorted by days descending then id ascending."""
    rows = []
    for inv in invoices:
        if inv.get("paid"):
            continue
        days = days_past_due(inv["due_date"], as_of)
        if days > OVERDUE_AFTER_DAYS:
            rows.append({"id": inv["id"], "vendor": inv["vendor"], "days": days,
                         "amount": Decimal(inv["amount"]).quantize(TWO_PLACES),
                         "risk": risk_by_vendor.get(inv["vendor"], "unknown")})
    rows.sort(key=lambda row: (-row["days"], row["id"]))
    return rows

def overdue_rows_to_json(rows: Iterable[Dict]) -> str:
    """CT-1: JSON array of {id, vendor, days (int), amount (two-decimal string), risk}; '[]' when empty."""
    return json.dumps([{"id": r["id"], "vendor": r["vendor"], "days": int(r["days"]),
                        "amount": str(Decimal(r["amount"]).quantize(TWO_PLACES)), "risk": r["risk"]} for r in rows])
```

Source: feat.design (B3.2 functions and algorithms); feat.trd (IF-5, IF-6).

4. Tests (new file `tests/test_overdue.py`)

The test file is new, as feat.tasks assigns, so the parallel lanes do not edit the same file. feat.design B3.5 allows either `tests/test_aging.py` or a new `tests/test_overdue.py`. feat.stories TK-05.2 had suggested `tests/test_aging.py` as agent judgement; the later feat.tasks split (TA-4) is followed here.

Test	Covers	Source
`test_overdue_boundary_30_and_31`	30 days not listed; 31 days listed with `days == 31`	feat.tasks (TSK-05); feat.design (B3.5); record:memory (L-007)
`test_overdue_excludes_paid_and_not_due`	Paid invoice 90 days past due not listed; not-yet-due not listed	feat.design (B3.5); feat.stories (AC-05.3, AC-05.4)
`test_overdue_row_keys_and_sort`	Keys exactly `id, vendor, days, amount, risk`; 75 before 42; tie on 42 gives `SYN-INV-002` before `SYN-INV-003`	feat.design (B3.5); feat.stories (AC-05.5, AC-05.6)
`test_overdue_unknown_risk`	Vendor missing from the map gives `'unknown'`	feat.design (B3.5); feat.stories (AC-05.7)
`test_overdue_amount_quantised`	`Decimal("980.5")` gives `Decimal("980.50")`, type `Decimal`	feat.design (B3.5); feat.stories (AC-05.8)
`test_overdue_missing_vendor_raises`	`KeyError` for an overdue unpaid invoice with no `vendor`	feat.tasks (TSK-05); feat.stories (AC-05.10)
`test_existing_aging_unchanged`	`aging_buckets`, `days_past_due`, `bucket_for` as today, including `bucket_for(30) == "31-60"`	feat.design (B3.5); feat.stories (AC-05.9); feat.trd (T-11)
`test_rows_to_json`	`"1250.00"` for SYN-INV-004; every `amount` a two-decimal string; `days` an integer; `[]` for no rows; key order; compare `json.loads` values	feat.tasks (TSK-06); feat.design (B3.5); feat.stories (AC-06.1 to AC-06.3)

Test data: `AS_OF = date(2026, 10, 25)`, as in the existing tests, with due dates built as `AS_OF - timedelta(days=N)` so that `days` is 30, 31, 42 and 75; synthetic ids and vendors only (feat.tasks TSK-05; tool:repo; feat.design SE-4). The CI bar requires every error branch to have its own test (feat.tasks section 2); the `KeyError` branch has one, and the other branches of BC-3 (paid skip, not-overdue skip, listed row, unknown risk) and BC-4 (empty and non-empty) are covered by the tests above.

5. Components not changed

Component	Why it stays as it is	Source
`BUCKETS`, `bucket_for`, `days_past_due`, `aging_buckets`	Must not change; existing signatures are kept and new functions are added	feat.tasks (TSK-05 "Must not change"); feat.trd (IF-5.8); feat.prd (NFR-4)
`tests/test_aging.py`, `tests/__init__.py`	Existing tests pass unchanged; new tests go to the new file	feat.tasks (section 2 "Test files"); tool:repo
CI command and gate	Unchanged: `/usr/bin/python3 ci.py`, coverage >= 90% and static checks	task.repo_access (section 2)
Transport or service between ap_aging and ap_portal	Not added; a transport would re-trigger DES-01	feat.tasks (TSK-06 "Must not change"); feat.design (DD-3, A8)
Database, schema, migration	None required	journey:task.db_changes; feat.design (A1)
Other lanes (ap_pipeline HC-1, HC-2; ap_forecast HC-4, HC-5; ap_portal HC-8; end-to-end harness HC-10)	Owned by the Data Engineering, Data Science, Frontend lanes and the Testing Agent; ap_aging imports none of them and consumes plain dicts (DC-1, DC-3)	feat.design (A2, A3, Part C "Lane dependencies")

6. Coding order and constraints for the coding step

1. TSK-05 first: BC-1 (`List`), BC-2, BC-3, BC-5 (`overdue_with_risk` export), BC-6, and its tests in BC-7.
2. TSK-06 next: BC-1 (`import json`), BC-4, BC-5 (`overdue_rows_to_json` export), and `test_rows_to_json`.
3. Constraints: Python 3.9 syntax only; no `print`, `eval`, `exec` or bare `except`; lines at most 140 characters; compile, unit tests, coverage of at least 90% and static checks green; commit messages start with the story ID (STORY-05 or STORY-06).

Sources: feat.tasks (sections 1, 2; TSK-05, TSK-06); task.repo_access (sections 1, 2); feat.stories (section 2).

7. Traceability

Component	Story acceptance criteria	PRD	TRD	Design
BC-2, BC-3	AC-05.1 to AC-05.10	AC-13 to AC-18	C-6; IF-5; T-8, T-9, T-10 (row part), T-11	HC-6; B3.2; DD-4, DD-10
BC-4	AC-06.1 to AC-06.4	AC-18; NFR-2	C-7; IF-6; T-10 (JSON part)	HC-7; CT-1; DD-3
BC-5, BC-6	Story tasks TK-05.1 (add to `__all__`), TK-05.4 (version step)	NFR-4	F-11; T-A6	B3.1; DD-11
BC-7	All of the above	NFR-8	T-8 to T-11	B3.5

Sources: feat.stories (STORY-05, STORY-06, section 7); feat.prd (section 3.4); feat.trd (sections 1, 2, 5); feat.design (A2, B3, D1).

8. Open questions (all non-blocking)

ID	Question	Interim answer	Owner (proposed)
CQ-BE-1	Which version number follows `1.4.0`?	One minor step, number confirmed by the lane in the coding step	Developer
—	Carried, unchanged: TQ-3 (CT-1 approval, AC-06.1 to AC-06.4 pending), TQ-9 (`KeyError` for missing `vendor`), TQ-1 (`bucket_for(30)`), DQ-1 (production wiring of CT-1), DQ-2 (rounding mode), PQ-2 (meaning of `days` and the `id` tie-break), RA-1 (branch name `feature/story`)	Interim designs as in feat.design, feat.tasks and task.repo_access	Architect for the TQ items (feat.trd 7.2); others as in their source

Sources: feat.tasks (section 2, section 7); feat.design (D3); feat.stories (section 3); task.repo_access (section 3).

9. Validation check

Check	Result	Source
Existing code read before listing changes	Pass (section 1)	tool:repo
Every component traces to a task, story and design element	Pass (sections 2, 7)	feat.tasks; feat.stories; feat.design
Change stays inside the TSK-05 and TSK-06 file budgets	Pass: `aging.py`, `__init__.py`, new `tests/test_overdue.py` only	feat.tasks (TSK-05, TSK-06 "Files in budget")
Existing signatures kept; only new functions added	Pass	feat.tasks (section 2); feat.prd (NFR-4)
No new technology, layer, transport or storage	Pass	feat.design (DD-1, DD-3); journey:task.db_changes
Decisions D-2026-031, D-2026-044, D-2026-050; lessons L-007, L-011	Consistent (ML-1, ML-2, ML-3, ML-5, BC-7)	record:memory
Conflict with an approved output	None. The test-file location differs between feat.stories TK-05.2 (agent judgement) and feat.tasks; feat.design B3.5 allows both, and the later feat.tasks split is followed	feat.stories (TK-05.2); feat.tasks (section 2); feat.design (B3.5)
Stop-and-ask ("a required dependency or database change is missing")	Not met: no database change is required, so none is missing	journey:task.db_changes (section 4)
Design and acceptance criteria clear enough to code	Yes: the design gives signatures, algorithms, error handling and tests for both functions; the pending criteria have interim designs	feat.design (B3.2 to B3.5); feat.tasks (TSK-05, TSK-06 "Acceptance criteria")
Blocking questions	None	—